

Parcial de Aprendizaje por Refuerzo

Grupo 5 — La estrategia de exploración

Santiago Pereira · Alexis Mesías · Nancy Mazariegos · Hugo Rivas · Brandon Reyes

1. Análisis del componente asignado

A nuestro grupo nos tocó revisar cómo explora el agente. El código original entrena con Q-learning y ϵ -greedy usando $\epsilon = 0.05$ fijo, 1000 episodios de 60 pasos cada uno, sobre un espacio de 200 estados (10 niveles de inventario $\times$ 5 niveles de días $\times$ 4 niveles de demanda) y 6 acciones, o sea 1200 pares (s,a) posibles. Todo lo que ponemos abajo sale de correr el código nosotros mismos con la semilla 42 que ya venía puesta.

Entregable 5.1 — Probabilidad de cubrir todos los pares (s,a) y el tema de la demanda crítica

Con $\epsilon = 0.05$ y 6 acciones, la probabilidad de que en un paso el agente pruebe una acción no greedy específica es $\epsilon/6 = 0.0083$. Si un estado s se visita n veces, la probabilidad de que una acción concreta de ese estado se pruebe al menos una vez es:

$$P(\text{par visitado}) = 1 - (1 - \epsilon/6)^n = 1 - (0.99167)^n$$

El problema es cuántas veces se visita realmente cada estado. Al correr el entrenamiento, las visitas por estado quedan así: el promedio es 300, pero la mediana es apenas 64 y hay estados con solo 2 visitas, mientras que el más visitado llega a 2664. O sea, el promedio engaña muchísimo porque el agente se queda pegado en unos pocos estados.

Con la mediana de 64 visitas, la probabilidad de probar una acción específica en ese estado es $1 - 0.99167^{64} = 0.41$. Como cada estado tiene 5 acciones que no son la greedy, y hay 200 estados, la probabilidad de cubrir los 1200 pares es el producto de todas esas probabilidades. Nos dio:

$$P(\text{cubrir los 1200 pares en 1000 episodios}) \approx 10^{-519}$$

Es decir, prácticamente cero. Y no es solo teoría: al medirlo, ϵ -greedy visitó 770 de 1200 pares (64.2%) y dejó 430 pares sin tocar ni una sola vez. Esto rompe directamente la condición de exploración infinita que pide la convergencia de Q-learning en las diapositivas, así que la Q de esos 430 pares se queda en 0 (el valor de inicialización) y nunca es una estimación de nada.

Sobre por qué los estados de demanda crítica salen tan mal parados, encontramos dos cosas en la función de transición:

Primero, la demanda nunca cambia dentro del episodio. En `step()` la línea es `new_dem = dem`, entonces el nivel de demanda con el que arranca el episodio es el mismo hasta el final. Los estados con demanda crítica solo se pueden alcanzar si el `reset()` los sortea, nunca se llega a ellos transicionando desde demanda baja, media o alta. Como `reset()` elige uniforme entre los 4 niveles, solo ~250 de los 1000 episodios empiezan en demanda crítica, y ahí se queda el agente atrapado en ese subespacio.

Segundo, los días tampoco cambian nunca. La transición hace `max(1, days-1)` y después discretiza al nivel más cercano de [1, 7, 14, 30, 60]. Como 59 se discretiza a 60, 29 a 30, 13 a 14 y 6 a 7, la variable días

se queda congelada en el valor inicial en todos los casos. Entre las dos cosas, lo único que se mueve en el estado es el inventario, y el espacio de 200 estados se parte en 20 cadenas independientes (una por cada combinación días × demanda) a las que solo se entra por reset.

Encima, en demanda crítica el consumo es de 40 unidades por día y la acción más grande es pedir 50, entonces el inventario se vacía rápido y el agente pasa casi todo el episodio en los mismos 2 o 3 niveles bajos de inventario. Por eso la cobertura por nivel de demanda queda tan despareja:

Nivel de demanda	Pares cubiertos	% cobertura	Sin visitar
bajo	210 / 300	70.0%	90
medio	218 / 300	72.7%	82
alto	192 / 300	64.0%	108
crítico	150 / 300	50.0%	150

Para ser exactos: no es que los estados de demanda crítica nunca se visiten (sí se visitan como estado inicial), es que la mitad de sus pares (s,a) nunca se prueba, que para efectos del aprendizaje es casi lo mismo. Justo son los estados donde equivocarse cuesta más caro.

Entregable 5.2 — Estrategia de exploración alternativa

Nosotros proponemos combinar inicialización optimista con UCB, en vez de seguir con ϵ -greedy o de subirle el ϵ .

La idea de la inicialización optimista es arrancar la tabla Q en un valor alto (nosotros usamos 100) en lugar de 0. Cada vez que el agente prueba una acción su valor se corrige hacia abajo respecto de ese valor inicial y las que no ha probado siguen viéndose mejores, así que el propio argmax lo obliga a probar todo al menos una vez. Es exploración dirigida y no depende de la suerte de un número aleatorio.

UCB agrega el bono de incertidumbre de las diapositivas:

$$a = \operatorname{argmax} [Q(s,a) + c \cdot \sqrt{(\ln N(s) / N(s,a))}]$$

donde $N(s)$ son las veces que se visitó el estado y $N(s,a)$ las veces que se tomó esa acción ahí. El bono es grande cuando una acción se ha probado poco en relación a su estado, y se va achicando conforme se acumula información. Usamos $c = 2.0$.

La justificación para el dominio farmacéutico es que en una farmacia explorar no es gratis. Con ϵ -greedy, el 5% de las veces el agente toma una acción totalmente al azar, sin importar si ya sabe perfectamente qué hacer en ese estado: puede pedir 0 unidades de un medicamento crítico que se está acabando solo porque salió el número. En un sistema real eso es un desabasto que le puede costar a un paciente. UCB nunca es aleatorio: es determinista y explora la acción que menos ha probado, así que la exploración se agota donde ya hay información y se concentra donde de verdad falta. La inicialización optimista además hace que el barrido de acciones nuevas ocurra al principio del entrenamiento, que es cuando se puede hacer en simulación antes de poner el modelo en producción.

Descartamos exploración por novedad porque necesita una función de novedad o un modelo adicional y agrega hiperparámetros que no tenemos cómo calibrar con este entorno. Y descartamos simplemente subir ϵ porque eso aumenta la exploración pero sigue siendo ciega: para cubrir los 430 pares faltantes

tendríamos que subir ϵ tanto que la política aprendida se degradaría, y de todos modos seguiría metiendo acciones aleatorias en estados críticos para siempre porque ϵ es fijo.

Entregable 5.3 — Implementación y métricas de cobertura comparativas

Implementamos la estrategia en la función `train_optimistic_ucb()`, con los mismos 1000 episodios, la misma semilla (42) y el mismo entorno, para que la comparación sea justa. Lo único que cambiamos fue el mecanismo de selección de acción y el valor inicial de Q; bajamos alpha de 0.9 a 0.1 porque con 0.9 la actualización se vuelve inestable y borra el efecto de la inicialización optimista en un solo paso.

La implementación completa de `train_optimistic_ucb()`, comentada línea por línea explicando la relación con las fórmulas de las diapositivas, se encuentra en el código adjunto.

Métricas de cobertura (espacio total: 200 estados $\times$ 6 acciones = 1200 pares):

Métrica	ϵ -greedy	Opt+UCB
Pares (s,a) visitados	770	1195
% de cobertura	64.2%	99.6%
Pares NO visitados	430	5
Estados visitados	200 / 200	200 / 200

Los 200 estados se visitan en las dos estrategias porque el `reset()` los sortea uniformemente; la diferencia real está en los pares (s,a), que es lo que importa para que Q converja.

Pares no visitados por nivel de demanda y por acción:

Nivel de demanda	ϵ -greedy	Opt+UCB
bajo	90	0
medio	82	0
alto	108	0
crítico	150	5

Acción	ϵ -greedy	Opt+UCB
pedir 0	4	0
pedir 10	78	0
pedir 20	88	0
pedir 30	82	1
pedir 40	89	2
pedir 50	89	2

Este cuadro nos pareció el más revelador: con ϵ -greedy la acción "pedir 0" se probó casi en todos los estados (solo 4 sin probar), mientras que las acciones de pedir cantidades grandes quedaron sin probar en ~80 estados cada una. Tiene sentido, porque la recompensa premia el inventario alto y castiga pedir (-2), entonces el agente se aferra rápido a no pedir y ϵ -greedy no le da suficientes oportunidades de descubrir lo contrario.

Distribución de visitas sobre los pares (s,a) y sobre los estados:

Visitas por par (s,a)	ϵ -greedy	Opt+UCB
0 visitas	430	5
1–5 visitas	389	1025
6–20 visitas	170	106
21–100 visitas	71	39
>100 visitas	140	25
Promedio (pares visitados)	77.9	50.2
Máximo	2407	3608

Visitas por estado	ϵ -greedy	Opt+UCB
1–50 visitas	96	165
51–200 visitas	26	10
201–500 visitas	36	0
>500 visitas	42	25
Promedio	300.0	300.0
Desviación estándar	461.0	783.1

Lo que leemos de estos números es que Opt+UCB pasa de 64.2% a 99.6% de cobertura, y sobre todo elimina por completo el hueco en demanda alta y crítica (de 108 y 150 pares sin visitar a 0 y 5). Lo que sí hay que decir es que la distribución no queda más pareja en el sentido clásico: la desviación estándar sube (461 a 783 por estado) y la mayoría de los pares nuevos quedan con solo 1 a 5 visitas. Eso es esperable, porque UCB garantiza que todo se pruebe al menos una vez y después vuelve a concentrarse en lo prometedor. Para efectos del problema esto está bien: tener 1 visita en un par ya rompe el $Q = 0$ heredado de la inicialización y da al menos una estimación, mientras que 0 visitas es no saber absolutamente nada. Los 5 pares que quedan sin visitar están todos en demanda crítica con acciones de pedir 30, 40 y 50, y creemos que es porque el inventario en esos estados se vacía tan rápido con demanda de 40/día que el episodio no alcanza a volver a pasar por esas combinaciones.

Entregable 5.4 — Relación entre la cobertura y los 23 stockouts semanales

Para conectar esto con producción buscamos los estados donde la política greedy que aprendió ϵ -greedy provoca un desabasto directo, o sea donde inventario + acción elegida < demanda diaria. Nos salieron 6 estados:

Estado	Acción elegida	Necesita	Acciones probadas
(10, 14, medio)	pedir o	15	5 / 6
(10, 14, alto)	pedir o	25	1 / 6
(10, 30, medio)	pedir o	15	1 / 6
(10, 30, alto)	pedir o	25	2 / 6
(30, 14, crítico)	pedir o	40	1 / 6
(30, 30, crítico)	pedir o	40	1 / 6

De esos 6 estados, 5 tienen exploración pobre (se probaron 3 acciones o menos de las 6). En esos 5 el agente eligió "pedir o" no porque haya comparado y concluido que era lo mejor, sino porque literalmente nunca probó las alternativas: las otras acciones seguían en $Q = 0$ y "pedir o" ganó el desempate. Es una decisión tomada sin información.

Si asumimos que cada estado que produce desabasto directo contribuye de forma parecida a los 23 stockouts de la semana, el reparto queda:

$$23 \times (5/6) \approx 19 \text{ stockouts por exploración insuficiente} \quad | \quad 23 \times (1/6) \approx 4 \text{ por otras causas}$$

O sea, alrededor de 19 de los 23 stockouts semanales (más o menos el 83%) son atribuibles directamente a pares (s,a) que nunca se visitaron durante el entrenamiento. Estos son los que sí se arreglarían mejorando la exploración: con Opt+UCB esos mismos estados tienen las 6 acciones probadas, entonces la elección de "pedir o" ya sería una decisión informada y no un default. Aclaremos que el 19 es un reparto proporcional bajo ese supuesto, no una medición directa; lo que sí está medido son los 6 estados y que 5 de ellos tienen exploración pobre.

Los otros ~4 no son culpa nuestra y queremos dejarlo claro: son del diseño de la recompensa. La función es $\text{reward} = \text{inventario} \times 0.5$, menos 10 si quedan pocos días, menos 2 si se pide algo, y no tiene ningún castigo por quedarse sin producto. El código hace $\max(0, \text{inv} + \text{accion} - \text{demanda})$, o sea que el desabasto se recorta a 0 en silencio y el agente ni se entera de que pasó. Aunque exploráramos el 100% de los pares, el agente aprendería la política correcta según esa recompensa, que igual no penaliza el stockout. Por eso decimos que la exploración explica la mayoría de los 23 pero no todos: la exploración determina qué tanto sabe el agente, y la recompensa determina qué es lo que aprende a querer.

2. Respuestas a las preguntas de integración

Grupos consultados y qué nos dio cada uno:

Grupo	Insumo recibido
Grupo 1	MDP corregido (mdp_corregido.py) y tabla de impacto 1.4
Grupo 2	Recompensa corregida 2.3 y tabla comparativa 2.4
Grupo 3	Algoritmo corregido, tabla de ablación y espacio real alcanzable
Grupo 4	preprocess_state mejorado y tabla de MSE 4.4
Grupo 6	Diagnóstico de curvas 6.1 y ablación por grupo
Grupo 7	Análisis de gap 7.1 / 7.2 y protocolo de evaluación 7.3

Pregunta 1 — Diagnóstico sistémico

Si se aplican únicamente las correcciones del MDP (Grupo 1) y de la recompensa (Grupo 2), sin tocar el algoritmo ni la exploración, ¿el agente aprenderá una política mejor? Nuestra respuesta es que no, y que incluso puede empeorar.

Primero, el Grupo 1 nos pasó su tabla de impacto: el estado pasa de 3 a 5 variables (agrega unidades en tránsito, 6 niveles, y tendencia de demanda, 3 niveles), y el espacio pasa de 200 a 3600 estados y de 1200 a 21600 pares (s,a). Es un factor de expansión de 18.

Segundo, ese factor cae directamente sobre nuestro componente. El entrenamiento son 1000 episodios de 60 pasos, o sea 60000 pasos. Con $\varepsilon = 0.05$, solo unos 3000 son exploratorios; los otros 57000 repiten la acción greedy, que en cada estado es una sola. Aunque cada paso exploratorio cayera en un par nuevo, el techo sería de unos 3600 pares por la parte greedy más 3000 por la exploratoria: 6600 de 21600, o sea 30% en el mejor caso imposible. Nosotros medimos 64.2% de cobertura sobre la rejilla declarada de 1200 pares; sobre el MDP corregido del Grupo 1, esa cobertura se desploma.

Tercero, la recompensa corregida del Grupo 2 solo puede enseñar algo en los estados que el agente visita. Su función hace que la mejor acción varíe por estado (en (0,1,crítico) la mejor acción es pedir 40, en (100,14,crítico) es pedir 0), que es exactamente lo que se necesita. Pero en un par (s,a) nunca visitado, Q sigue en 0 sin importar qué tan buena sea la recompensa, porque nunca se ejecuta la actualización. Una señal mejor sobre estados que no se visitan no aporta nada.

El Grupo 6 lo midió empíricamente en su ablación. Comparando solo las configuraciones que mantienen la recompensa original, que son las únicas comparables entre sí porque cambiar la recompensa cambia la escala del retorno:

Configuración	Reward (últimos 20 ep.)
Baseline (original)	289.62
Solo G1 (MDP corregido)	-10.70
Solo G3 (algoritmo corregido)	93.03
Solo G5 (exploración corregida)	590.70

Aplicar solo el MDP corregido baja el retorno de 289.62 a -10.70. Es coherente con nuestro análisis: se agranda el espacio 18 veces sin darle al agente ninguna forma nueva de recorrerlo. Aclaramos que las configuraciones del Grupo 6 que incluyen la recompensa del Grupo 2 (-627.73 para "Solo G2" y -636.60

para todas combinadas) no son comparables contra el baseline, porque están medidas con otra función de recompensa; no las usamos como evidencia.

Conclusión: corregir MDP y recompensa sin corregir exploración produce un agente que sabe menos que antes sobre un mundo más grande. El orden importa.

Pregunta 2 — Causa raíz

Entre el error de implementación del algoritmo, la cobertura insuficiente y el reward hacking, ¿cuál contribuye más al gap? Nuestra respuesta es la cobertura insuficiente, con el desempate del argmax como amplificador.

El reward hacking es real pero no explica los stockouts. El Grupo 2 demostró que bajo la recompensa original $R(\text{pedir } 50) - R(\text{pedir } 0)$ es positivo en los 220 estados probados, con un mínimo de +0.5, o sea que pedir el máximo domina siempre. Pero cuando simularon esa política de pedir 50, el resultado fue **stockout promedio = 0**. El reward hacking produce los 41 vencimientos semanales y el costo de 8400 contra el objetivo de 3200 ($2.63\times$), no los 23 stockouts. El propio Grupo 2 lo dejó escrito: no asignar los 23 stockouts únicamente a la recompensa.

Y el reward hacking ni siquiera llegó a materializarse. El Grupo 6 corrió el código original con un env reconstruido y midió la política greedy real: pedir 0 en 79.2% de los estados y pedir 50 en apenas 5.3%. El comentario del enunciado que reporta "pedir 50 en 94% de estados" no reproduce. El agente no alcanzó la política que la recompensa premiaba.

Se quedó ahí porque no exploró. Nuestro dato: 430 de 1200 pares (s,a) nunca visitados (35.8%), y en demanda crítica 150 de 300, la mitad. El Grupo 3, midiendo sobre el espacio realmente alcanzable, obtuvo una cobertura de 1370.8 de 26226 pares, un 5.23%.

El mecanismo exacto lo imprimió el Grupo 7 en su entregable 7.2: un estado no visitado conserva $Q(s) = [0, 0, 0, 0, 0, 0]$, y np.argmax sobre un vector de ceros devuelve el índice 0, que en la lista de acciones es pedir 0 unidades. Es la peor acción posible justo donde falta producto. El Grupo 7 además encontró que (0, 1, crítico) con acción 0 transiciona a sí mismo: es un estado absorbente de desabasto.

El Grupo 3 cuantificó cuánto pesa ese desempate con su ablación C1: cambiando únicamente el criterio de desempate a aleatorio, sin tocar nada más, el valor capturado sube de 30.63% a 68.46% y el acierto de política de 6.82% a 22.1%.

Ablación (Grupo 3)	Valor capturado	Cobertura (s,a)
Original	30.63%	5.23%
C1 desempate aleatorio	68.46%	4.58%
C2 alpha con decaimiento	33.58%	5.39%
C3 epsilon con decaimiento	73.11%	8.36%
C4 identidad on-policy (SARSA)	33.81%	5.38%
C5 inicialización optimista	30.74%	6.71%
Corregido (todas)	75.88%	8.21%

Esto matiza nuestra propia posición y queremos decirlo: el desempate del argmax es formalmente un error de implementación, no de exploración. Pero los dos son la misma falla vista desde dos lados. La

cobertura insuficiente es la que produce los empates en cero, y el desempate es el que los convierte sistemáticamente en "pedir o". Sin cobertura pobre no hay empates que desempatar; sin el desempate sesgado, los pares no visitados no colapsarían todos hacia la misma acción mala.

Nuestro entregable 5.4 cierra el número: de los 6 estados donde la política greedy provoca desabasto directo, 5 tienen 3 acciones probadas o menos, lo que atribuye alrededor de 19 de los 23 stockouts semanales a pares nunca visitados.

Sobre el error de implementación del Grupo 3 (el código se presenta como SARSA pero calcula Q-learning, porque usa argmax sobre el siguiente estado), lo consideramos un problema real de corrección teórica pero de menor impacto en este gap: su propia ablación C4 sube el valor capturado de 30.63% a solo 33.81%, contra el 68.46% del desempate y el 73.11% del decaimiento de ϵ .

Pregunta 3 — Plan de corrección mínimo viable

Proponemos cinco cambios en este orden. El criterio de priorización es ganancia medida sobre esfuerzo, usando las ablaciones del Grupo 3 y del Grupo 6 como evidencia.

#	Cambio	Componente	Métrica que mejora	Días
1	Desempate aleatorio en argmax	Algoritmo (G3)	Valor capturado 30.63% $\rightarrow$ 68.46%	1
2	Opt+UCB + decaimiento de ϵ	Exploración (G5, G3)	Cobertura 64.2% $\rightarrow$ 99.6%	3
3	Recompensa corregida	Recompensa (G2)	Recompensa -842.29 $\rightarrow$ +144.40	2
4	MDP corregido	Estado/transición (G1)	Restaura Markov; 200 $\rightarrow$ 3600 estados	3
5	Protocolo de evaluación	Evaluación (G7)	Compuerta de cobertura pre-despliegue	2

Prioridad 1. Es una línea de código y es el cambio de mayor retorno individual de todo el plan: valor capturado de 30.63% a 68.46% y acierto de política de 6.82% a 22.1% (ablación C1 del Grupo 3).

Prioridad 2. Nuestra propuesta lleva la cobertura de pares de 64.2% a 99.6% sobre la rejilla declarada, y en demanda crítica de 50% a 98.3%; el decaimiento de ϵ del Grupo 3 (C3) lleva la cobertura sobre el espacio real de 5.23% a 8.36% y el valor capturado a 73.11%. Recomendamos combinar ambas y no solo una: el Grupo 3 probó inicialización optimista sola (C5) y el valor capturado apenas se movió, de 30.63% a 30.74%, lo que indica que el optimismo por sí solo no basta y que el trabajo lo hace el mecanismo de selección. Va en segundo lugar porque habilita todo lo demás: sin cobertura, ninguna corrección de recompensa o de MDP puede propagarse a la tabla Q.

Prioridad 3. Métricas del entregable 2.4 del Grupo 2, comparando pedir siempre 50 contra su política corregida: recompensa media de -842.29 a +144.40, pasos en ventana de vencimiento de 18.11 a 7.48, unidades pedidas de 1500 a 582.94, e inventario promedio de 97.98 a 5.76. Ataca los 41 vencimientos y el costo de 2.63× el objetivo, que son los síntomas que la exploración no explica.

Prioridad 4. Necesario para modelar el mundo real, pero multiplica el espacio por 18. Va después de la prioridad 2 por lo que argumentamos en la Pregunta 1: el Grupo 6 midió que aplicarlo solo baja el retorno de 289.62 a -10.70. Incluye recalibrar el esquema de decaimiento de ϵ para el espacio nuevo.

Prioridad 5. La capa 1 del protocolo del Grupo 7 exige cobertura de estados $\geq 95\%$, cobertura de pares (s,a) $\geq 90\%$, y mínimo 80% dentro de cada categoría de demanda incluyendo crítico, antes de siquiera mirar métricas de recompensa. Aplicando ese criterio al sistema auditado: la estrategia original reprueba (64.2% global, 50% en crítico) y nuestra propuesta aprueba (99.6% global, 98.3% en crítico). Este es el cambio que habría evitado el despliegue original.

Total estimado: 11 días de trabajo, dentro de las dos semanas. El orden no es negociable entre la prioridad 2 y la 4: invertir las reproduce el resultado que el Grupo 6 ya midió.

Pregunta 4 — Compatibilidad de correcciones

Revisamos el código de los tres grupos y encontramos una incompatibilidad real, una dependencia y un detalle que rompe en ejecución.

Incompatibilidad: Grupo 1 con Grupo 4. El estado del Grupo 1 tiene 5 componentes (inventario, días, demanda, unidades en tránsito, tendencia). La función `preprocess_state` del Grupo 4 desempaqueta exactamente 3 valores, así que recibir un estado de 5 la rompe. El Grupo 1 previó el problema e incluyó `estado_a_original()`, que proyecta de 5 a 3 variables, y con eso el preprocesamiento del Grupo 4 corre sin tocarse. Pero esa proyección descarta justo las dos variables nuevas, que son las que restauran la propiedad de Markov. O sea, la solución de compatibilidad anula el aporte del Grupo 1.

Nuestra propuesta para resolverla: extender el vector de características del Grupo 4 de 12 a 16, agregando `en_transito` normalizado (dividido entre 50), tres indicadores one-hot para la tendencia (bajando, estable, subiendo), y opcionalmente una característica de inventario proyectado que sume inventario más unidades en tránsito, que es la cantidad que de verdad determina si habrá desabasto mañana.

Dependencia: Grupo 3 requiere a Grupo 4. En su tabla de triada mortal, el Grupo 3 midió que con características sin normalizar el entrenamiento divergió en 5 de 5 corridas, tanto off-policy como on-policy, alrededor del episodio 48 a 50. Con características normalizadas divergió en 0 de 5 y capturó entre 87.5% y 98.4% del valor. El Grupo 4 confirma la causa desde su lado: en el aproximador original el componente de inventario aporta el 74.9% de la norma y el de demanda solo el 0.09%, con un número de condición de 1888.2 que baja a 26.3 al normalizar. No son correcciones opcionales entre sí: el algoritmo corregido del Grupo 3 solo es estable si se aplica el preprocesamiento del Grupo 4.

Detalle que rompe en ejecución: la tilde en "crítico". El código original y el MDP corregido del Grupo 1 usan la cadena 'crítico' con tilde. El Grupo 4 trabaja internamente con 'critico' sin tilde y lo resuelve con una función auxiliar que quita el acento antes de indexar. Mientras se llame en ese orden funciona, pero cualquier código que compare las cadenas directamente, o que use las claves del Grupo 4 contra el diccionario del Grupo 1, va a fallar con `KeyError`. Recomendamos fijar una sola convención y normalizar en un único punto de entrada.

Advertencia sobre el preprocesamiento mejorado. El Grupo 4 reporta que su versión mejorada reduce el MSE de prueba de 990.07 a 678.40 frente al original con sesgo, y sube el R^2 de 0.952 a 0.967. Pero su propio número de condición sube de 26.3 en la versión normalizada simple a 2130.5 en la mejorada, o sea peor que el original sin normalizar. Dado lo que el Grupo 3 midió sobre divergencia y mal condicionamiento, sugerimos verificar la estabilidad del entrenamiento con el vector mejorado antes de adoptarlo, o regularizar las características que introducen la colinealidad.

Resumen: las tres correcciones son compatibles siempre que se aplique el orden 4 antes que 3, se extienda el preprocesamiento del Grupo 4 para no descartar el estado del Grupo 1, y se unifique la convención de la tilde.

3. Reflexión grupal

Cuando empezamos, nuestro diagnóstico era que el problema era $\epsilon = 0.05$, o sea que el valor estaba muy bajo y por eso el agente no exploraba lo suficiente. Nuestra primera idea fue subirle el epsilon o ponerle un decay y listo. Después de meternos en el código nos dimos cuenta de que la exploración no era el único problema, sino que estaba encadenada con todo lo demás.

Lo que más nos cambió el diagnóstico fue el Grupo 2. Nosotros dábamos por hecho que la exploración explicaba los 23 stockouts y también parte del sobreabastecimiento. Ellos simulaban la política de pedir 50 y les dio stockout promedio igual a cero, y demostraron que bajo la recompensa original pedir el máximo domina en los 220 estados. Eso partió el problema en dos mitades limpias: la recompensa explica los 41 vencimientos y el costo de 2.63 veces el objetivo, y nosotros explicamos los 23 stockouts. Dejamos de pelear por atribuirnos todo el gap y el argumento quedó más fuerte, no más débil.

Lo segundo que nos cambió fue el mecanismo. Sabíamos que 430 pares quedaban sin visitar y que su Q se quedaba en cero, pero no habíamos cerrado el paso siguiente. El Grupo 7 lo dejó explícito en su 7.2: un estado no visitado tiene $Q(s) = [0, 0, 0, 0, 0]$ y np.argmax de puros ceros devuelve el índice 0, que es pedir 0 unidades. El Grupo 3 lo midió con su ablación de desempate: cambiar solo eso sube el valor capturado de 30.63% a 68.46%. Entramos pensando que teníamos un problema de exploración y salimos entendiendo que teníamos un problema de exploración amplificado por un desempate sesgado, y que la corrección más barata del plan completo no era la nuestra sino una línea del Grupo 3.

Lo tercero fue de humildad con nuestros propios números. El Grupo 3 y el Grupo 4 midieron por separado que el espacio realmente alcanzable no son los 200 estados que declara el encabezado sino alrededor de 4370, porque la transición no discretiza el inventario. Nuestro 64.2% de cobertura está medido sobre la rejilla declarada de 1200 pares; el Grupo 3, midiendo sobre el espacio real, reporta 5.23%. No son números contradictorios, son denominadores distintos, pero nos obligó a decir con precisión sobre qué espacio estábamos midiendo en vez de dar el porcentaje como si fuera absoluto.

Y lo último fue el orden. Al principio habríamos dicho que corregir el MDP y la recompensa era lo primero, porque son los componentes más grandes. El Grupo 6 midió que aplicar solo el MDP del Grupo 1 baja el retorno de 289.62 a -10.70, y que aplicar solo la exploración lo sube a 590.70. Corregir el modelo del mundo antes de corregir cómo se recorre hace que el agente sepa menos sobre un mundo más grande. La conclusión con la que nos quedamos es que la exploración define cuánto sabe el agente y la recompensa define qué quiere, pero que ninguna de las dos sirve si el orden de aplicación está invertido.